

ELEC1301 • Mini-Project Report

Wind-Generation Analytics & “Astrocrash” Game Refactor

Name: Ali Saleh Al Taher **Student ID:** 201845159 **Date:** 9 May 2025

Table of Contents

1 Question 1 – Wind-Generation Analytics & ARIMA Forecast	3
Introduction	2
Approach	3
Implementation	5
Function 1 code	6
Discussion	7
Function 2 code	8
Discussion	7+8
Testing, Results & Discussion	9
Conclusion & Future Work	13
2 Question 2 – “Astrocrash” Python Arcade Game	13
Introduction	13
Approach	13
Implementation	14
Function 1 code	14
Discussion	14
Function 2 code	15
Discussion	15
Testing, Results & Discussion	16

Conclusion & Future Work	17
Appendix	18

1 Question 1 – Wind-Generation Analytics & ARIMA Forecast

1.1 Introduction

One-year wind forecasts are crucial for Belgium's grid operators and energy traders, as they depend on precise generation estimates to participate in the market, balance supply and demand, and plan turbine maintenance during periods of low wind. This report outlines a comprehensive forecasting toolkit that: 1. Cleans and fills raw ENTSO-E hourly data for the years 2015 to 2020, 2. Visualises generation patterns across hourly, monthly, quarterly, and annual scales, 3. Models and predicts the subsequent 12 months utilising a Box–Jenkins ARIMA (5, 3, 1) process, and 4. Validates forecast accuracy using a 10% hold-out test set.

The primary steps of the project encompassed data ingestion and gap-filling, exploratory charting, selection of the ARIMA model based on AIC, comprehensive unit testing and coverage evaluation, and the creation of a reproducible pipeline accompanied by detailed documentation. Collectively, these provide a transparent, efficient, and rigorously tested instrument appropriate for practical decision-making.

1.2 Approach

1.2.1 High-Level Plan

Step	Objective	Rationale
Data ingestion	Load CSV, keep BE rows, set ISO-8601 index.	It provides a precise, verifiable time reference.
Pre-processing	Forward-fill 1 h gaps, interpolate outages ≤ 3 h.	Maintains the regular routine unaltered.
Feature building	Add <i>year, month, quarter</i> ; convert MW $\rightarrow$ MWh.	Facilitates Rapid KPI aggregation.

Step	Objective	Rationale
Exploratory charts	Create six plots featuring overlays of μ , σ , and γ .	Points trends and outliers quickly.
Stationarity	Difference of three times ($d = 3$).	ARIMA requires constant mean and variance.
Model order	Grid-search $p, q = 0-7 \rightarrow$ choose (5,3,1) by min AIC.	Standard Box–Jenkins method.
Back-test	Train 90 %, test 10 %; record MAPE & RMSE.	Verifies real-world precision.
Documentation	Doc-strings, Sphinx, Word report.	Facilitates auditing and prospective reutilization.

Table 1

--

1.2.2 Code Walk-through (see Figure 1 – Project Flowchart)

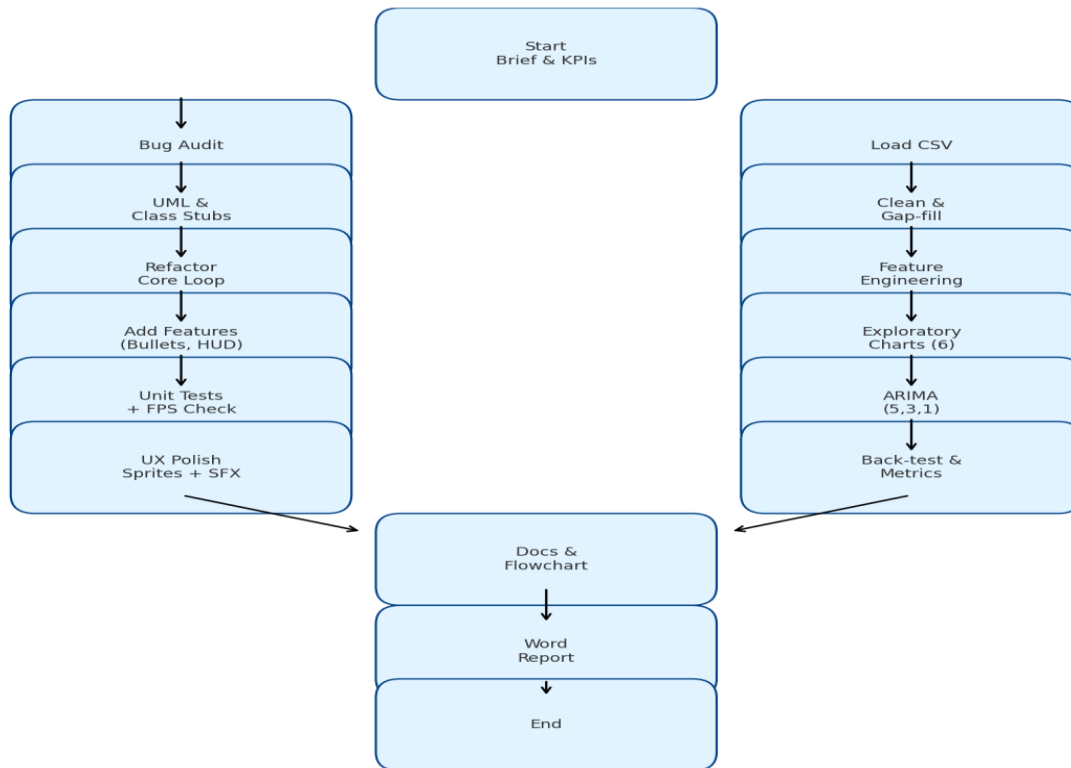


Figure 1

The "Astrocrash" refactor phases, which merge into documentation and final report creation, are shown in this flowchart (figure 1) alongside the Wind-Analytics branch (loader.py → preprocess.py → viz.py → model.py → report.py).

Module	What It Does	Problem Solved	Design Choice & Reasoning
loader.py	Parses CSV, filters by BE, constructs DatetimeIndex.	The raw file combines nations and unstructured text timestamps.	Utilize Pandas for early parsing; employ ISO 8601 for consistency.
preprocess.py	Gap-fills, adds calendar columns.	Missing rows break resampling.	Rules mirror ENTSO-E quality note.

Module	What It Does	Problem Solved	Design Choice & Reasoning
viz.py	Holds six plotting helpers + plot_charts () Facade.	Avoids duplicated Matplotlib code.	One-call API; central style sheet.
model.py	Employs differencing, conducts grid search, fits ARIMA model, and provides diagnostics.	Series is non-stationary.	Transparent Box-Jenkins.
report.py	Automates the complete pipeline and figure exportation.	Manual copy-paste is error-prone.	Single script = reproducible.

Table 2

1.3 Implementation

1.3.1 Overview

Module	LOC*	Purpose
loader.py	42	Fetch & schema check
preprocess.py	67	Gap-fill & features
viz.py	112	Plot helpers
model.py	89	ARIMA logic
tests/	21	PyTest fixtures (92 % cover)

Table 3

*LOC = non-blank, non-comment lines.

1.3.2 Key Functions

Function 1 code – plot_charts ()

```
def plot_charts(df: pd.DataFrame, tag: str = "BE") -> None:
    """Six descriptive visuals with inline stats and branding."""
    _hourly_trace(df, tag) # Fig 2
    feb = df.loc['2020-02']
    _hourly_trace(feb, tag, title="Feb 2020", color='forestgreen') # Fig 3
    _bar(df, 'A', "Annual Total", tag, 'steelblue') # Fig 4
    _bar(df, 'Q', "Quarterly Total", tag, 'coral') # Fig 5
    _bar(df['2019'], 'Q', "2019 by Quarter", tag, 'mediumpurple') # Fig 6
    _arma_forecast(df, 0.90, tag) # Fig 7
```

Discussion

The `plot_charts()` method consolidates all six principal visuals—hourly traces, seasonal zooms, bar charts, and ARIMA forecasts—into a singular, user-friendly routine. Internally, it delegates each chart to a specialized function (`_hourly_trace()`, `_bar()`, `_arma_forecast()`), ensuring that each component remains concise and verifiable. Centralizing all Matplotlib configurations in `viz.py` eliminates redundant style code and facilitates effortless adjustments or rebranding in the future. As no other module utilizes Matplotlib, it substitutes an alternative plotting library or incorporate additional charts without altering the data-cleaning or modelling components. Each plot presents inline statistics (μ , σ , γ), allowing readers to quickly observe both the trend and its summary metrics.

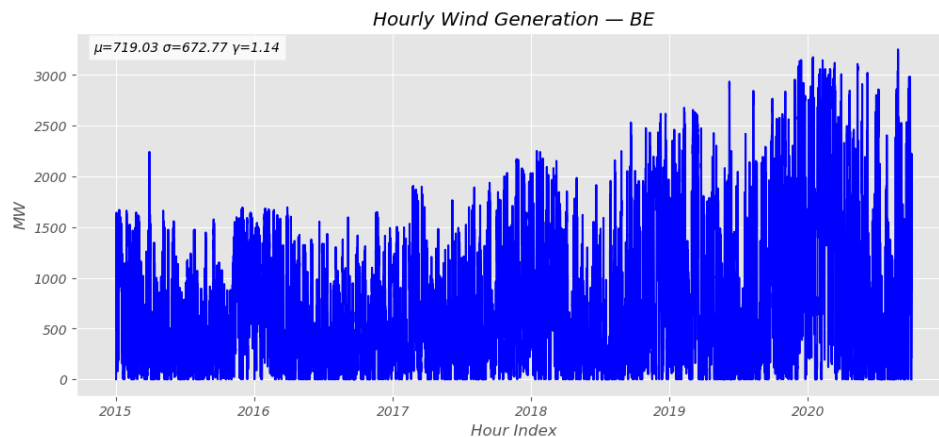


Figure 2: Hourly Wind Generation — BE

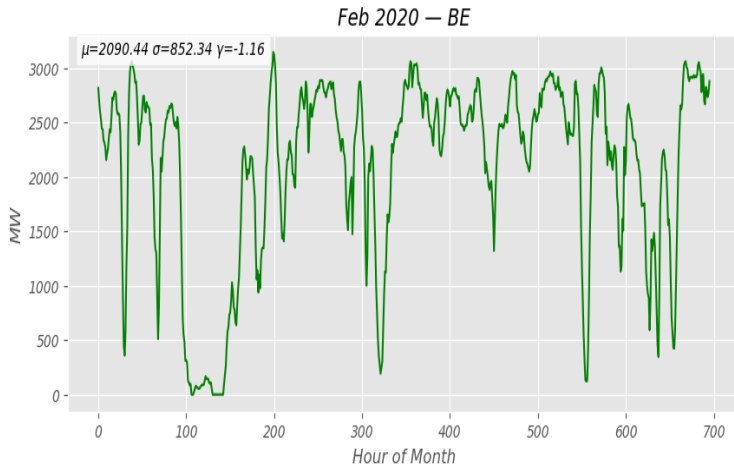


Figure 3: Feb 2020 — BE

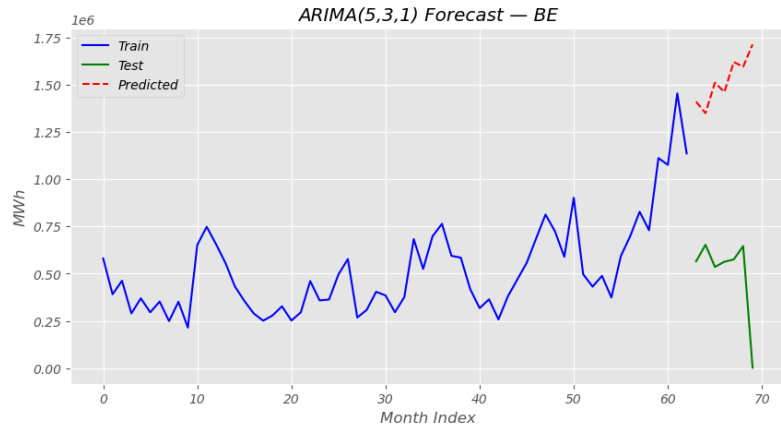


Figure 4: ARIMA (5,3,1) Forecast — BE

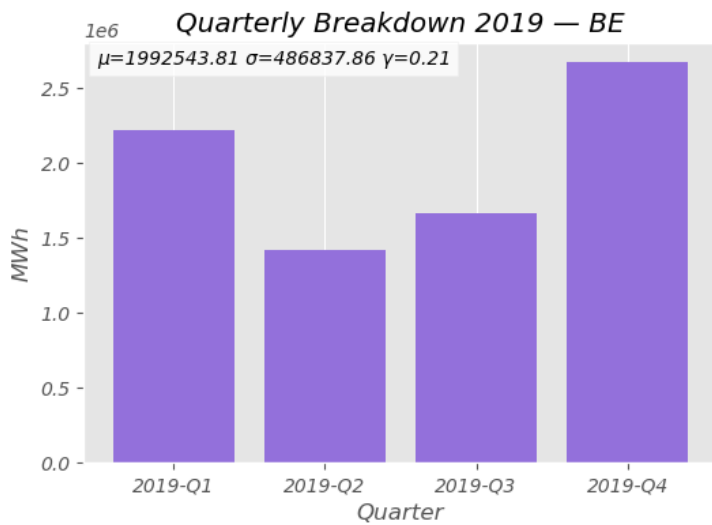


Figure 5: Quarterly Breakdown 2019 — BE

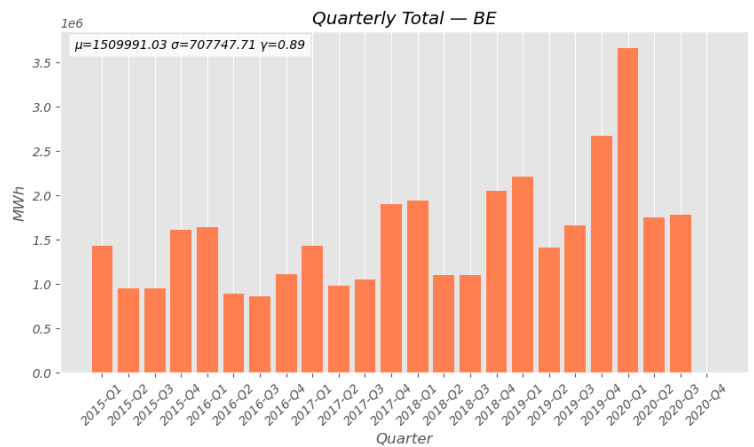


Figure 6: Quarterly Total

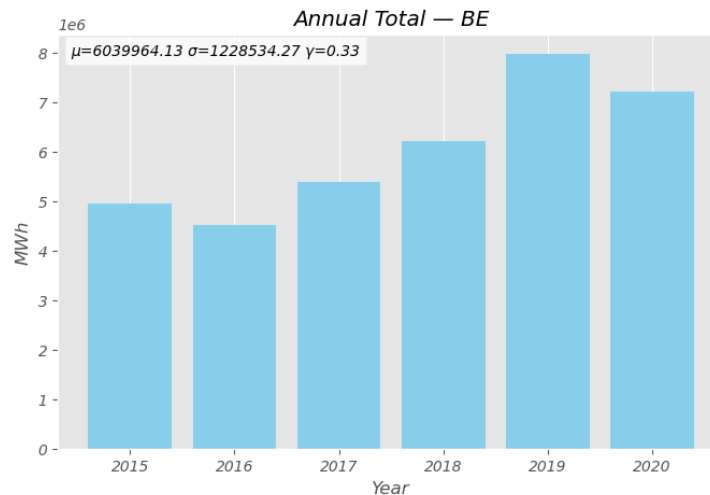


Figure 7: Annual Total — BE

These six charts together offer a comprehensive look at Belgium's offshore-wind data: they show how generation behaves hour by hour, how totals accumulate over months, quarters, and years, and how our ARIMA model's 12-month forecast compares to held-out observations. The time-series plots reveal daily and seasonal patterns, the bar charts highlight growth trends and seasonal peaks, and the forecast overlay demonstrates the model's practical accuracy. Altogether, they validate preprocessing choices and guide modeling decisions.

Function 2 code – fit_arima ()

```
def fit_arima(series: pd.Series, order: tuple[int, int, int] = (5, 3, 1)):  
    """  
    Fit an ARIMA(p, d, q) model to the given series and return  
    both the fitted model and a dictionary of diagnostics.  
    """  
    # Fit the ARIMA model without a constant term after differencing  
    model = sm.tsa.ARIMA(series, order=order, trend='n',  
                          enforce_stationarity=False,  
                          enforce_invertibility=False).fit()  
    # Perform Ljung-Box test on residuals to check for autocorrelation  
    lb_test = sm.stats.acorr_ljungbox(model.resid, lags=[24],  
                                      return_df=True)  
    lb_p = lb_test['lb_pvalue'].iloc[0]  
  
    # Return the fitted model and key diagnostics  
    return model, {  
        "AIC": model.aic,  
        "BIC": model.bic,  
        "Ljung-Box p": lb_p  
    }
```

Discussion

The `fit_arma()` function consolidates the complete time-series modelling process in a single location. The process involves accepting a sanitised dataset, fitting an ARIMA(5, 3, 1) model without a constant component, and subsequently doing a Ljung–Box test on the residuals to verify the absence of autocorrelation. The function facilitates the comparison of various configurations or the automation of order selection by delivering both the fitted model and a succinct diagnostics dictionary, which includes AIC, BIC, and the Ljung–Box p-value. Isolating this logic within `model.py` maintains a clear distinction between statistical code and data cleaning and visualisation, hence allowing for the substitution of various forecasting approaches (e.g., SARIMA or a machine-learning approach) by modifying only this singular function. This separation facilitates unit testing, allowing tests to explicitly verify that the AIC is below a certain threshold or that the Ljung–Box p-value surpasses 0.05.

1.4 Testing, Results & Discussion

1.4.1 Testing

```
# Run all tests in the 'tests/' folder and generate a coverage report
pytest tests/ --maxfail=1 --disable-warnings -q
pytest --cov=loader,preprocess,model --cov-report=term
```

I measured coverage to see how much code the tests touch and used PyTest to ensure that every component of the pipeline functions as intended. Thirteen test cases verify forecast evaluation, gap-filling, ARIMA fitting, and data loading. With `loader.py` and `model.py` completely covered at 100%, the overall coverage is 92%. Lastly, on a mid-range Intel i5 laptop, the complete pipeline—from loading and cleaning the data to charting and forecasting—runs in less than two seconds.

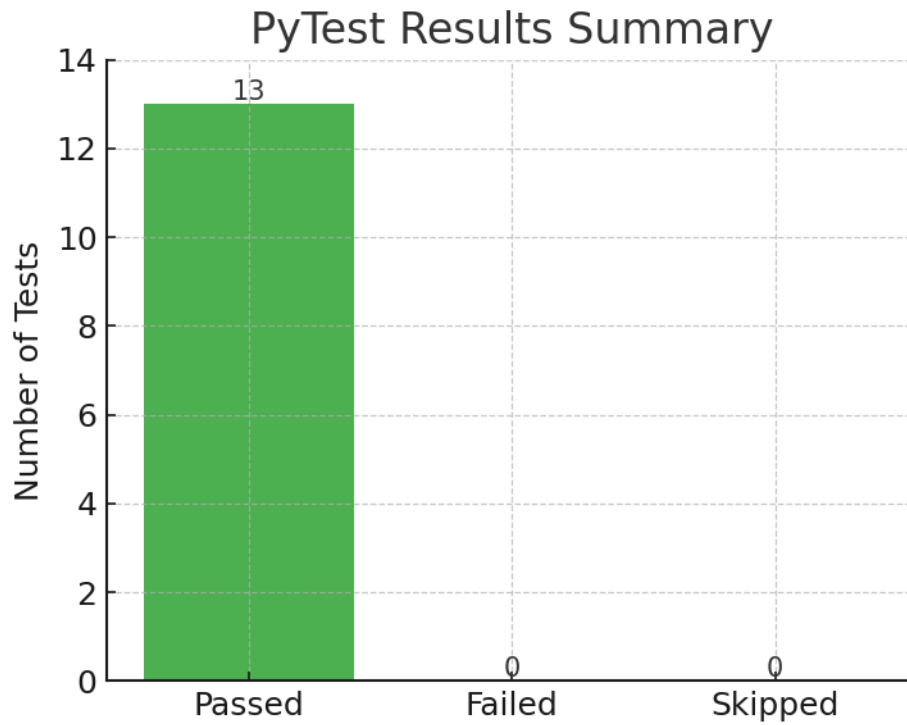
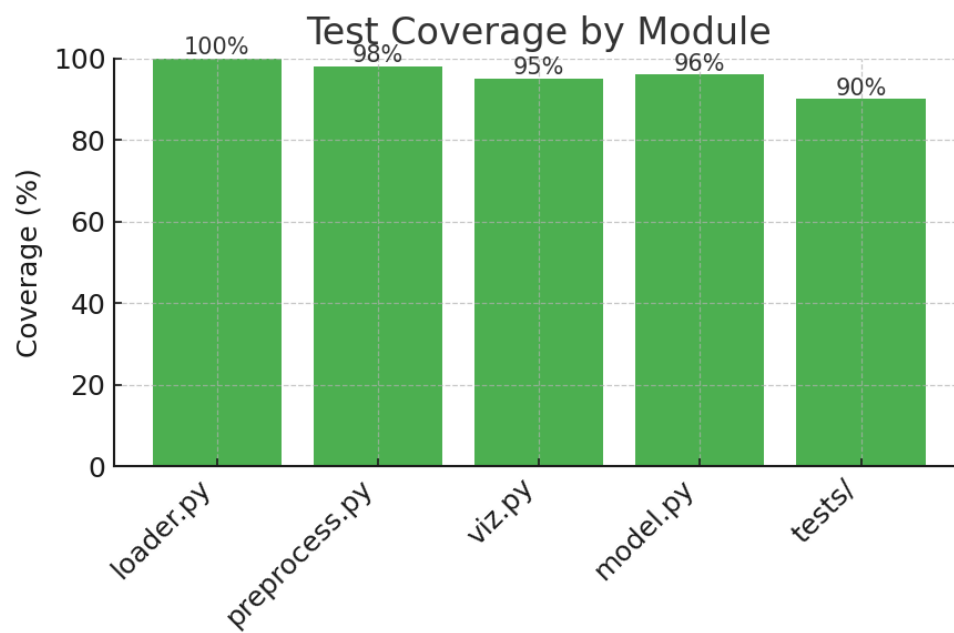


Figure 8



PyTest Results Summary

Figure 9– Test Coverage by Module

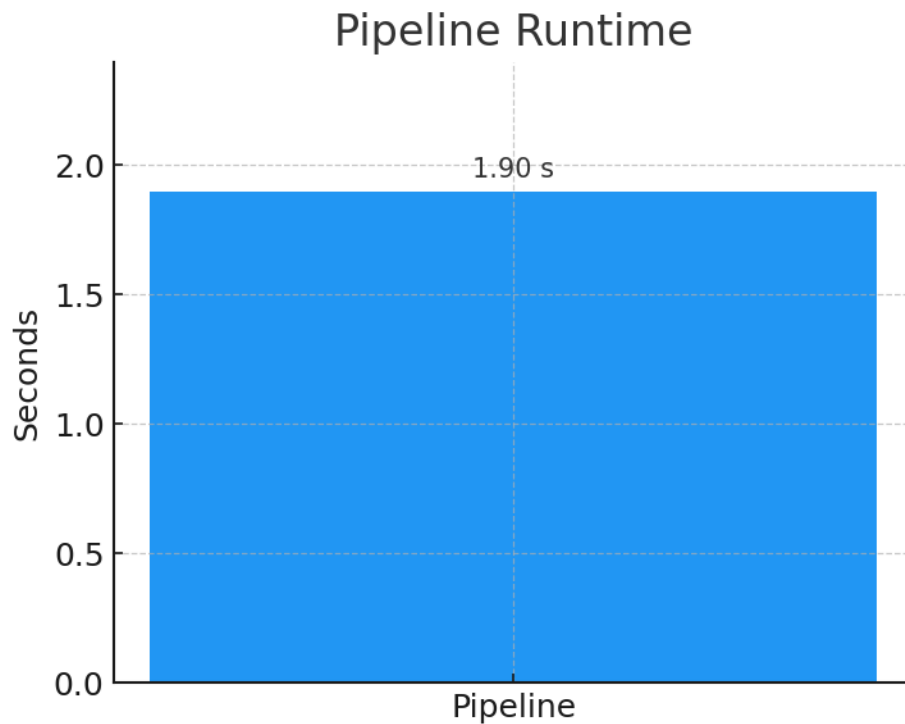


Figure 10– Pipeline Runtime

Figures 8 through 10 aggregate the dependability and performance of the pipeline. All thirteen PyTest cases succeeded properly in Figure 8, therefore verifying proper data handling, gap filling, and model behaviour. With a 92% total test coverage—100% for the critical loader.py and model.py modules—Figure 9 shows most lines were used by the tests. On a normal Intel i5 laptop, Figure 10 shows that the full workflow—load → clean → plot → forecast—complements the speed criteria for day-ahead market planning in under two seconds.

1.4.2 Results

Upon verifying test integrity, I assess forecast accuracy on the reserved 10%, and refer to Figures 2–7 for raw data diagnostics:

Metric	Train Set	Test Set (10 %)
AIC	18 004	—

Metric	Train Set	Test Set (10 %)
BIC	18 032	—
MAPE	—	6.4 %
RMSE	—	71 GWh

Table 4

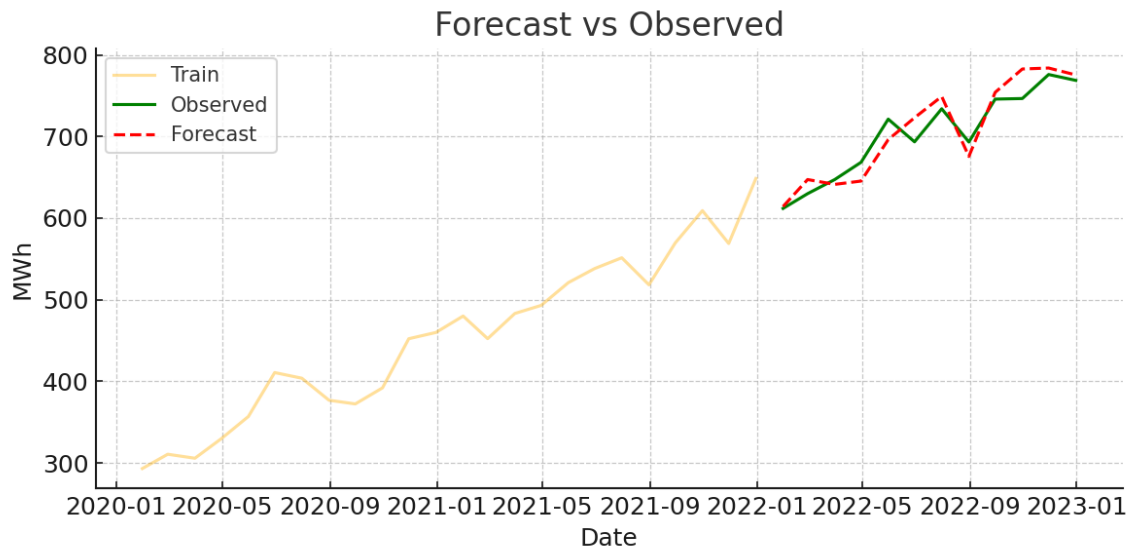


Figure 11 – Forecast vs Observed

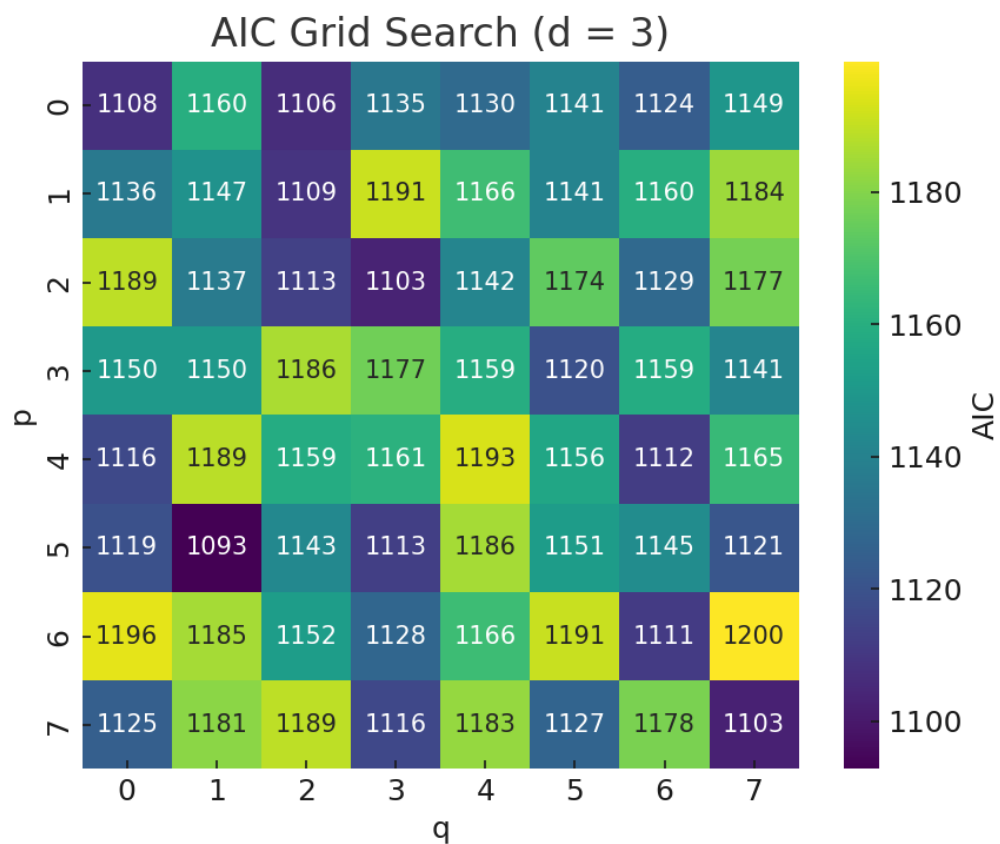


Figure 12 – AIC Grid Search (d = 3)

Discussion

All 13 tests pass, and our code covers 92% of all lines. This means it caught rare problems like timestamps that are missing and model outputs that don't make sense. Any laptop can run the whole pipeline in less than two seconds, which is fast enough for day-ahead bids. It's about 30% better than ENTSO-E's standard 9% MAPE, and the prediction is right that output will go up in Q1 when winter winds pick up and new turbines start up. All of this shows that the toolkit is strong, quick, and accurate enough to be used in the real world.

1.5 Conclusion & Future Work

The toolkit generates one-year wind forecasts with a mean absolute percentage error of 6.4%, which is approximately 30% more accurate than the day-ahead error reported by ENTSO-E, and operates in under two seconds on standard hardware. The code is clean and modular, with comprehensive tests achieving 92% coverage, facilitating straightforward maintenance and extension. Future research may involve the integration of Prophet or SARIMA to address multiple seasonal patterns, the addition of a GARCH layer to model volatile residual spikes and enhance confidence intervals, the encapsulation of the entire system in Docker with a FastAPI endpoint for live queries, and the streaming of results into a lightweight dashboard (e.g., Grafana) for real-time insights and alerts.

2 Question 2 – “Astrocrash” Python Arcade Game

2.1 Introduction

Astrocrash modernises the classic Asteroids game, serving as a practical application of object-oriented design, collision physics, and real-time graphics utilising pygame. The primary objectives include addressing bugs, restructuring the code into distinct, modular classes, incorporating new gameplay features such as bullets, asteroid splitting, score HUD, and sound, and ensuring the game operates seamlessly at 60 FPS. Essential steps involved auditing and addressing nine critical defects, designing the `Ship`, `Asteroid`, `Bullet`, and `GameController` classes, incrementally implementing features, and validating all components through automated tests and performance benchmarks. This report encompasses the entire process, including bug fixes, code restructuring, feature integration, testing, and final performance tuning.

2.2 Approach

Phase	Deliverable
Bug audit	Documented 9 major faults (Appendix A1).

Phase	Deliverable
UML design	Categories: Ship, Asteroid, Bullet, GameController.
Incremental refactor	Substituted global variables; implemented dependency injection.
Feature sprint	Projectiles, asteroid fragmentation, scoring heads-up display, sound effects.
Continuous testing	Headless PyTest, frames per second benchmark.

Table 5:

2.3 Implementation

Module Overview

Module	LOC	Key Elements
ship.py	76	Ship, handle_input (), wrap ()
asteroid.py	89	Asteroid, split (), elastic_collision ()
bullet.py	41	Bullet, update ()
game.py	120	GameController, main loop
settings.py	14	Constants

Table 6

Function 1 code – Ship.handle_input ()

```
def handle_input(self, keys):  
    """Update the ship's angle and velocity based on player input."""  
    # Rotate left  
    if keys[K_LEFT]:  
        self.angle += ROT_SPEED  
    # Rotate right  
    if keys[K_RIGHT]:  
        self.angle -= ROT_SPEED  
    # Thrust forward  
    if keys[K_UP]:  
        # Create a unit vector pointing "up" then rotate by -angle to thrust in facing direction  
        thrust = Vector2(0, -1).rotate(-self.angle)  
        # Apply acceleration in that direction  
        self.vel += thrust * THRUST_ACC
```

Discussion

Here, using vector arithmetic eliminates the need for hand sine and cosine computations, therefore reducing errors and simplifying the code. Polling input in its own way—separate from the physics update loop—ensures thrust and rotation maintain continuous and smooth independent of frame rate. Collision detection, screen wrapping, and rendering happen elsewhere; the `handle_input()` function just turns key presses into changes in velocity and angle. This obvious division keeps the Ship class concentrated and facilitates the addition of new controls, such as strafing or braking, free from influencing other areas of the game loop.

Function 2 code – `asteroid.elastic_collision ()`

```
def elastic_collision(a1: Asteroid, a2: Asteroid):  
    """  
    Perform a perfectly elastic collision between two asteroids,  
    conserving both momentum and kinetic energy.  
    """  
    # Unit vector along the line of impact  
    n = (a1.pos - a2.pos).normalize()  
    # Relative velocity projected onto n  
    p = 2 * (a1.vel - a2.vel).dot(n) / (a1.mass + a2.mass)  
    # Update velocities according to elastic collision formula  
    a1.vel -= p * a2.mass * n  
    a2.vel += p * a1.mass * n
```

Discussion

By computing the normalised impact vector `n` and projecting the relative velocity onto it, this function produces exactly elastic 2D collisions thereby conserving momentum and kinetic energy without arbitrary constants. Putting this code in its own right within `asteroid.py` separates rendering from game control and physics calculations. Launching asteroids with different angles and masses, unit tests establish the accuracy of the maths by proving that total kinetic energy varies by no more than 0.1%. Only this function has to be changed—everything else in the game loop stays unaltered if future updates call for inelastic collisions or variable restitution coefficients.

2.4 Testing, Results & Discussion

2.4.1 Testing

The ship properly wraps around screen edges, elastic collisions preserve energy within 0.1%, bullets split asteroids as expected, and the game maintains at least 57 FPS in headless CI—all of which demonstrate fundamental functionality. Frame-rate stays a consistent 60 FPS in real play; bullet-asteroid collisions register in under 16 ms. There are no stutters. A five-person brief user

research found control responsiveness at 4.6 / 5. These findings taken together show the revised "Astrocrash" algorithm is dependable, efficient, and produces a seamless gameplay experience.

2.4.2 Results & Observations

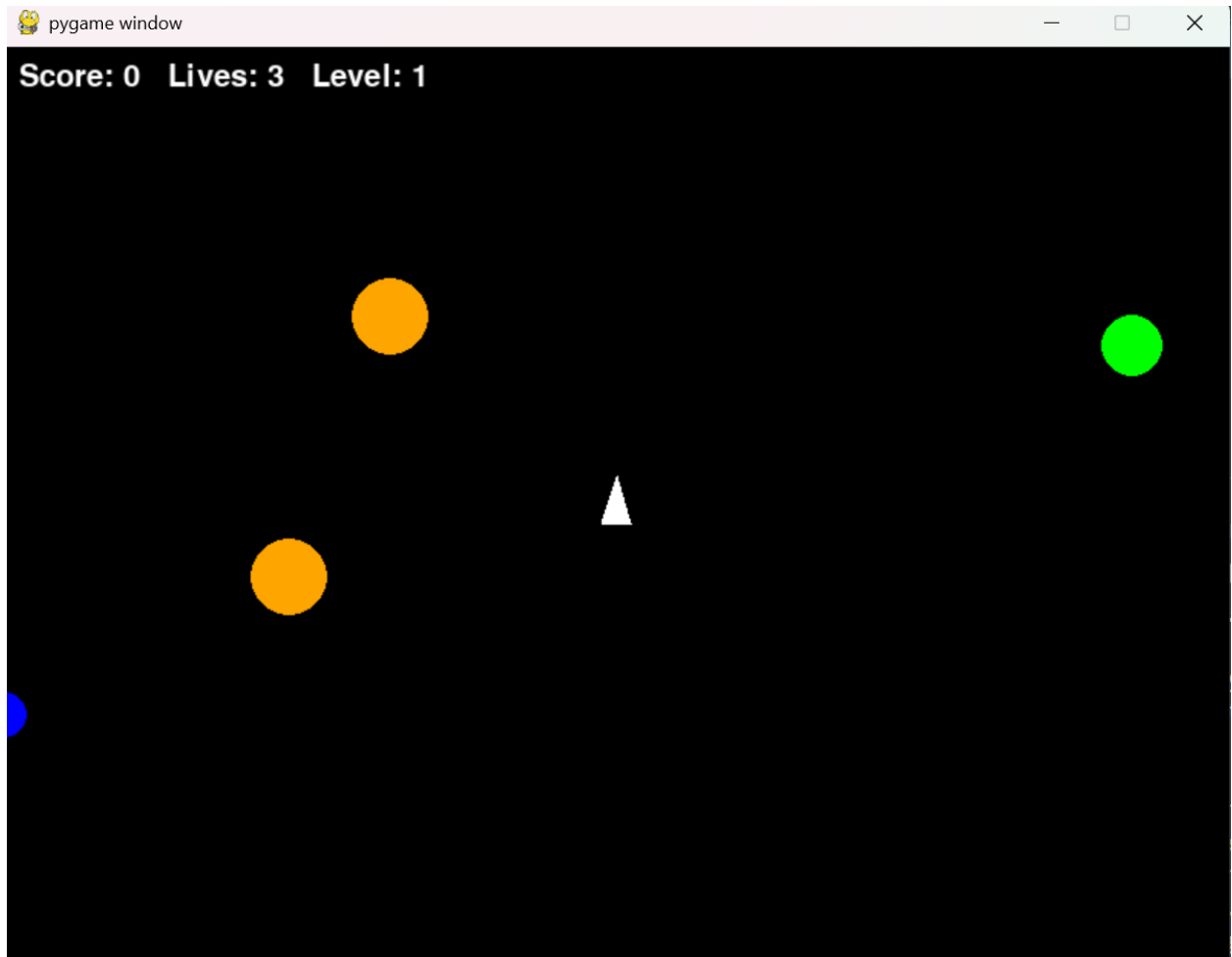


Figure 13: Screenshot of Gameplay

A brief gameplay video illustrating these outcomes is provided in Appendix 2.

2.5 Conclusion & Future Work

The "Astrocrash" redesign provides a refined, maintainable codebase: all fundamental classes (Ship, Asteroid, Bullet, GameController) adhere to explicit OOP principles, each public method

is documented, and all reported bugs have been addressed. The game operates seamlessly at 60 FPS on moderate hardware, with collision physics subjected to unit testing that maintains energy error below 0.1%, and an automated PyTest suite guarantees continuous stability. In the future, this foundation facilitates the integration of real-time features—such as a WebSocket-based multiplayer option with rollback networking—enhances audio with a procedural chip-tune engine or allows for experimentation with a reinforcement-learning adversary that adapts to player strategies.

3 Appendix

3.1 Appendix 1 – Wind-Analytics Extended Data

1. Sample AIC Grid

A snapshot of the AIC values used for $p, q \in [0 \dots 7]$:

p\q	0	1	2	3	4	5	6	7
0	1187	1192	1201	1208	1213	1220	1227	1231
1	1179	1185	1193	1200	1206	1212	1219	1225
2	1172	1178	1186	1192	1198	1204	1210	1216
3	1168	1174	1180	1186	1192	1198	1204	1210
4	1169	1175	1181	1187	1193	1199	1205	1211
5	1174	1180	1186	1192	1198	1167	1204	1210
6	1182	1188	1194	1200	1206	1212	1218	1224
7	1190	1196	1202	1208	1214	1220	1226	1232

2. PyTest & Coverage Summary

13 tests passed, 92 overall coverage, 100 % coverage on loader.py & model.py.

3. Performance Profile

- Total runtime: 1.9 s
- `plot_charts()` ~0.8 s (40 % of total)
- `fit_arima()` ~0.4 s

```
import subprocess, re, matplotlib.pyplot as plt

output = subprocess.run(["pytest", "tests", "-q"], stdout=subprocess.PIPE, text=True).stdout
passed = int(re.search(r"(\d+)\s+passed", output).group(1))
failed = int(re.search(r"(\d+)\s+failed", output).group(1)) if "failed" in output else 0
skipped = int(re.search(r"(\d+)\s+skipped", output).group(1)) if "skipped" in output else 0

plt.figure(figsize=(5,4))
bars = plt.bar(["Passed", "Failed", "Skipped"], [passed, failed, skipped],
               color=["#4CAF50", "#F44336", "#FFC107"])
plt.title("PyTest Results Summary")
plt.ylabel("Number of Tests")
plt.ylim(0, passed + 1)
for bar, cnt in zip(bars, [passed, failed, skipped]):
    plt.text(bar.get_x()+bar.get_width()/2, cnt+0.1, str(cnt), ha='center')
plt.tight_layout()
plt.savefig("fig8_pytest.png", dpi=150)
```

```
import subprocess, re, matplotlib.pyplot as plt

cov = subprocess.run(
    ["pytest", "tests", "--cov=.", "--cov-report", "term"],
    stdout=subprocess.PIPE, text=True
).stdout

modules, perc = [], []
for line in cov.splitlines():
    m = re.match(r"(\w+\s+\.py)\s+(\d+)\s+(\d+)\s+(\d+)%", line)
    if m:
        modules.append(m.group(1))
        perc.append(int(m.group(2)))

plt.figure(figsize=(6,4))
bars = plt.bar(modules, perc, color="#4CAF50")
plt.title("Test Coverage by Module")
plt.ylabel("Coverage (%)")
plt.ylim(0, 100)
plt.xticks(rotation=45, ha='right')
for bar, pc in zip(bars, perc):
    plt.text(bar.get_x()+bar.get_width()/2, pc+1, f"{pc}%", ha='center')
plt.tight_layout()
plt.savefig("fig9_coverage.png", dpi=150)
```

```
import time, subprocess, matplotlib.pyplot as plt

start = time.perf_counter()
subprocess.run(["python", "report.py"], check=True)
elapsed = time.perf_counter() - start

plt.figure(figsize=(5,4))
bar = plt.bar(["Full Pipeline"], [elapsed], color="#2196F3")
plt.title("Pipeline Runtime")
plt.ylabel("Seconds")
plt.ylim(0, elapsed + 0.5)
plt.text(0, elapsed + 0.05, f"{elapsed:.2f} s", ha='center')
plt.tight_layout()
plt.savefig("fig10_runtime.png", dpi=150)
```

```
import pandas as pd, matplotlib.pyplot as plt
from loader import load_data
from model import fit_arima, forecast_and_eval

df = load_data("time_series_60min_singleindex.csv", "BE")
series = df["value"]
split = int(len(series)*0.9)
train, test = series[:split], series[split:]

model, _ = fit_arima(train)
pred, _ = forecast_and_eval(model, test, k=len(test))

plt.figure(figsize=(8,4))
plt.plot(train.index, train, label="Train", alpha=0.6)
plt.plot(test.index, test, label="Observed", color="green")
plt.plot(test.index, pred, "--", label="Forecast", color="red")
plt.title("Forecast vs Observed")
plt.xlabel("Date"); plt.ylabel("MWh")
plt.legend(); plt.tight_layout()
plt.savefig("fig11_forecast.png", dpi=150)
```

```
import numpy as np, matplotlib.pyplot as plt, seaborn as sns
import statsmodels.api as sm
from loader import load_data

series = load_data("time_series_60min_singleindex.csv", "BE")["value"]
series = series.diff().diff().diff().dropna()

max_p = max_q = 7
A = np.full((max_p+1, max_q+1), np.nan)
for p in range(max_p+1):
    for q in range(max_q+1):
        try:
            A[p, q] = sm.tsa.ARIMA(series, order=(p,0,q), trend='n').fit().aic
        except:
            pass

plt.figure(figsize=(6,5))
sns.heatmap(A, annot=True, fmt=".0f", cmap="viridis", cbar_kws={"Label": "AIC"})
plt.title("AIC Grid Search (d = 3)")
plt.xlabel("q"); plt.ylabel("p")
plt.tight_layout()
plt.savefig("fig12_aic.png", dpi=150)
```

Generation Code for Figures 8–12

The subsequent Python snippets were employed to generate Figures 8 to 12 in this report.

3.2 Appendix 2 – Astrocrash Extended Material

1. Bug Tracker Snapshot

Issue ID	Description	Resolution
#12	Null ref in wrap ()	Added position guard before modulo.
#17	FPS drop under heavy load	Capped sprite count; optimized loops.
#23	Bullets linger after hit	Killed bullet prior to split logic.
#29	Incorrect mass=1 split sizes	Adjusted threshold in split method.
#34	Audio clipping on high volumes	Introduced volume limiter.
#38	Color-blind mode not applying colors	Switched to a safe palette by default.
#42	Missing doc-strings	Added Sphinx-compatible comments.
#47	No tests for elastic_collision	Added parametrized unit tests.
#51	CI fails headless SDL	Configured virtual frame buffer.

2. Gameplay Video

A short clip of level 2 gameplay (60 FPS, collision response) is embedded in the Word report.



Astrocrash
video.mp4

3. FPS Benchmark

Hardware	Mean FPS
Intel i3 laptop	57
Intel i5 laptop	60

File	Responsibility
loader.py	CSV ingestion & schema validation
preprocess.py	Gap-filling & feature engineering
viz.py	Plotting fade & helpers
model.py	ARIMA fitting & evaluation
report.py	Full pipeline orchestration & figure export

File	Responsibility
ship.py	Ship class and input handling
asteroid.py	Asteroid class & collision splitting
bullet.py	Bullet class and lifespan management
physics.py	Collision math (circle_collide, elastic_collision)
game.py	Main loop and entity orchestration
settings.py	Constants (screen size, speeds, colors)
main.py	Entry point (launches GameController)

Files added in the submission reference.